

JAVASCRIPT DOCUMENT

- JavaScript provides three different value comparison operations: strict equality using ===, loose equality using ==, and the [Object.is\(\)](#) method.
- A **closure** is when a **function remembers** the **variables** from where it was **created**, even after that place is **gone**.

EXAMPLE:

```
function outer() {  
  let name = "Manu";  
  function inner() {  
    console.log("Hello " + name);  
  }  
  return inner;  
}  
  
const greet = outer(); // outer() runs and returns inner  
greet(); // Output: Hello Manu
```

1. outer() finishes running — normally, its variable name would disappear.
2. But the inner function **inner()** **remembers** name because of **closure**.
3. So even after outer() is done, inner() still has access to name.

- A **closure** is: “A function that remembers variables from its outer function, even after the outer function has finished.”
- **Inheritance** means one object or class can **use the properties and methods** of another.
It helps you **reuse code** instead of writing the same logic again and again.

Example:

```
class Animal {  
  
  eat() {  
  
    console.log("This animal eats food");  
  
  }  
  
}
```

```

class Dog extends Animal {

  bark() {

    console.log("The dog barks");

  }

}

const dog = new Dog();

dog.eat(); // Inherited from Animal

dog.bark(); // From Dog

```

Here:

- Animal is the **parent class** (or **superclass**)
- Dog is the **child class** (or **subclass**)
- Dog **inherits** eat() from Animal
- Prototype chain: In JavaScript, **every object** has a hidden property called `[[Prototype]]` (or accessible via `__proto__`), which **links it to another object**. That “another object” is its **prototype**.

Example:

```

function Animal(name) {

  this.name = name;

}

Animal.prototype.eat = function() {

  console.log(this.name + " is eating");

};

function Dog(name, breed) {

  Animal.call(this, name); // Inherit properties

  this.breed = breed;

}

```

```
// Inherit methods using the prototype chain

Dog.prototype = Object.create(Animal.prototype);

Dog.prototype.constructor = Dog;

Dog.prototype.bark = function() {
  console.log(this.name + " barks!");
};

const myDog = new Dog("Tommy", "Labrador");

myDog.eat(); // inherited from Animal

myDog.bark(); // from Dog
```

1. Animal.prototype has the eat() method.
2. We set Dog.prototype = Object.create(Animal.prototype) — this means Dog.prototype **inherits** from Animal.prototype.
3. When myDog.eat() is called:

- JS doesn't find eat in myDog
- It goes up the **prototype chain** → finds it in Animal.prototype.

- The chain looks like: myDog → Dog.prototype → Animal.prototype → Object.prototype → null

Chain	Prototype	The object another object inherits from
	Prototype	A series of links connecting objects through inheritance
	Object.create()	Used to set up inheritance manually
	constructor	Points back to the function that created the instance

- Equality operators: The result of evaluating an equality operator is always of type boolean based on whether the comparison is true.
 - ==
Equality operator.
 - !=
Inequality operator.
 - ===
Strict equality operator.
 - !==
Strict inequality operator.
-
- Garbage Collection :
Garbage Collection (GC) is the process where JavaScript **automatically frees memory** that your program no longer needs.
We need garbage collection because whenever we create values javascript allows memory for that but that memory is limited so the unnecessary data must be removed.
- This decides what to delete by the rule called Reachability :
1. If your value is reachable i.e still used in the code , it stays in memory that doesn't gets deleted.
 2. If a value becomes unreachable i.e stopped using the value in code, then it gets deleted automatically.

REACHABILITY : STILL HAS A PATH IN THE CODE. THE VALUE ARE STILL IN THE PATH.

Examples of reachable values:

- Global variables
- Variables inside functions currently running
- Values referenced by reachable objects

- MARK AND SWEEP ALGORITHM:

1. Mark Phase

The GC starts from **root objects** (things always reachable):

- Global variables
- window object (in browser)
- Call stack variables
- Closures currently in use

GC checks what these root objects reference.

Then it follows those references and **marks all reachable objects**.

Example:

```
let a = { x: 1 };
```

```
let b = { y: 2 };
```

```
let c = { z: 3 };
```

```
a.next = b;
```

Reachable:

- a (because it is a variable)
- b (because a.next points to it)

Unreachable:

- c (nothing points to it)

✓ GC marks a and b

✗ c stays unmarked

2. Sweep Phase

After marking is done:

- GC scans the entire memory
- Anything **not marked** is considered “garbage”
- These unmarked objects are deleted/freed

In our example:

a → reachable

b → reachable

c → unreachable → removed

Putting It Together

Step 1: Start from roots

GC starts at global variables.

Step 2: Mark all reachable objects

Follow references → mark everything alive.

Step 3: Sweep everything else

Delete unmarked objects.

Phase	What Happens
Mark	Mark all reachable objects starting from roots
Sweep	Delete everything that wasn't marked

- **async/await :**
`async` makes a function return a promise.
`await` pauses the function until the promise resolves

Topic	Key Idea
• async/await vs Promises	async/await is cleaner and easier
• Using async in loops	use <code>for...of</code> or <code>Promise.all()</code>
• Using async inside classes	class methods can be async to handle APIs, DB, etc.

- Closure:
A **closure** happens when:

1. **A function** is created **inside another function**
2. The inner function **remembers and can use the variables** of the outer function
3. **Even after the outer function has finished running**

1. What is a closure in JavaScript and why is it useful?

A **closure** is when a function **remembers** the variables from the place where it was created **even after that outer function has finished running**.

Why is it useful?

Closures allow you to:

- keep data **private**
- create **function factories**
- maintain **state** without global variables
- build **timers, counters, caches, modules**, etc.

Short definition (perfect for interviews):

A closure is the combination of a function and its lexical environment.

1. What is Lexical Scoping? (Simplest Explanation)

Lexical scoping means:

A variable is accessible **based on where it is written in the code**,

NOT based on where the function is called.

In other words:

The structure of your code decides what variables you can use.

2. How let and const fit into Lexical Scope

let and const are **block scoped**.

This means:

They only exist inside:

- { } braces
- loops
- if blocks
- functions

var is NOT block scoped — it ignores { } and becomes function scoped.

- **Lexical Scoping**

Scope is decided by where code is written.
Inner functions remember outer variables.

- **let & const**

Block scoped → new variable per block.
Work better with loops + closures.

- **var**

Function scoped → one variable for whole function.
Causes closure bugs in loops.

- **Temporal Dead Zone (TDZ)**

TDZ = the period between:
when a variable is hoisted
but **before** the line where it is initialized
During TDZ, using the variable causes an error.

- **Standard Built-in Objects in JavaScript**

JavaScript provides many objects **built into the language**. They are available globally without importing anything.

1. Fundamental Objects

These are the most basic building blocks.

Object	Purpose
Object	Base of all objects.
Function	Every function is a Function object.
Boolean	Wraps true/false values.
Symbol	Unique identifiers.

2. Numbers & Math

Object	Purpose
Number	For numeric values.
BigInt	For very large integers.
Math	Provides math functions (sqrt, random...).
Date	Represents dates and times.

3. String & Text

Object	Purpose
String	Text handling.
RegExp	Regular expressions (pattern matching).

4. Indexed Collections

These store items by index.

Object	Purpose
Array	Standard list.
TypedArray	Binary data arrays.
ArrayBuffer	Raw memory buffer.

5. Keyed Collections

Store data in key-value format.

Object	Purpose
Map	Key-value pairs (any key type).
WeakMap	Keys are weakly referenced (no memory leaks).

Object	Purpose
Set	Unique values.
WeakSet	Objects stored weakly.

6. Structured Data

Object	Purpose
JSON	Parse/stringify JSON.
Math	Structured math operations.

7. Control Abstractions

Object	Purpose
Promise	Asynchronous operations.
GeneratorFunction	Used with generators (function*).
AsyncFunction	Used with async functions.

8. ERROR Objects

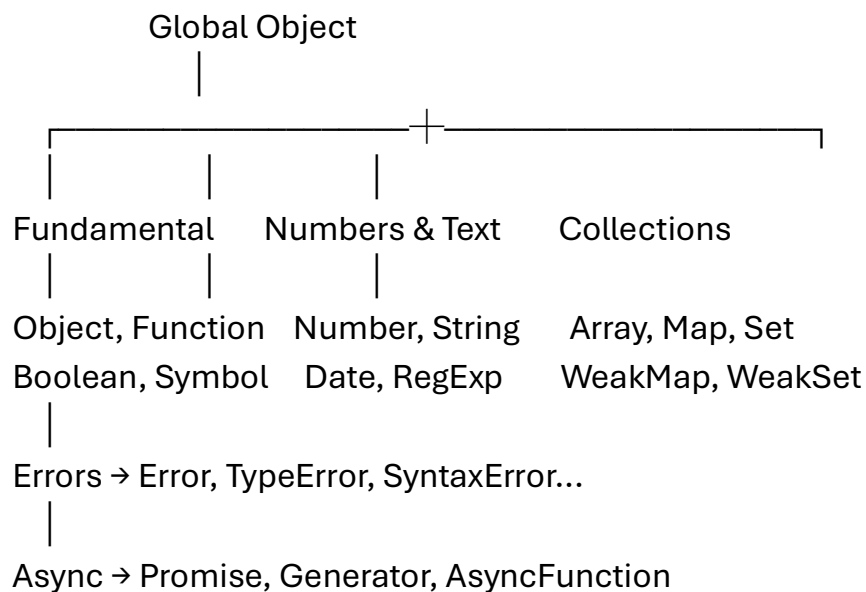
Object	Purpose
Error	Base error.
TypeError	Wrong type used.
ReferenceError	Variable not defined.
SyntaxError	Invalid syntax.
RangeError	Out-of-range values.
URIError	Incorrect URI encoding.

9. Web APIs (Browser Only)

Not part of JavaScript core, but used every day.

Object	Purpose
Window	Browser window.
Document	DOM access.
localStorage	Store data in browser.
fetch	HTTP requests.

DIAGRAM (Easy to Remember)



- **AggregateError:**
It groups multiple errors together into one error object

Why does it exist?

Because sometimes **many promises run together**, and you want:

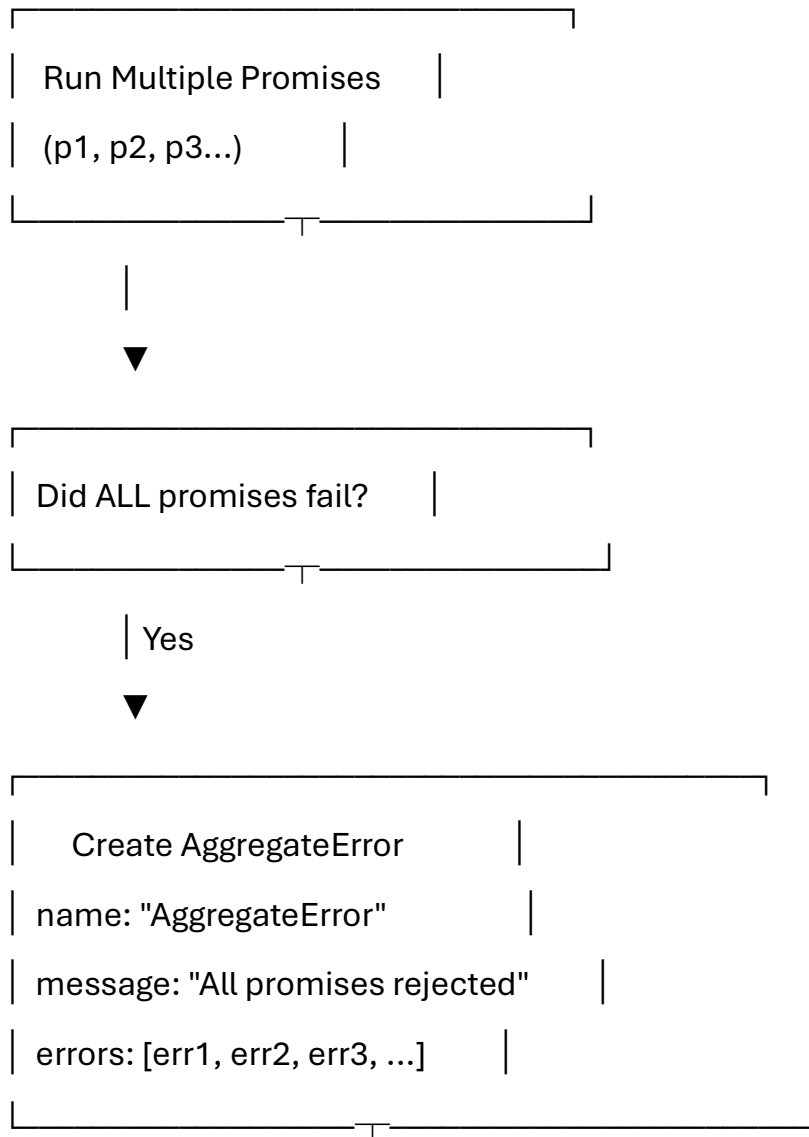
- ONE error to represent all failures
- Still be able to inspect **each individual error**

- **Why Is AggregateError Useful?**

Because in real projects:

- When calling **multiple APIs**, all may fail
- When validating a form, there may be **many validation errors**
- Parallel tasks may all throw errors

Instead of throwing 10 separate errors, you can throw **one clean grouped error**.





Throw ONE error containing ALL errors

- **Constructor:**
A function used to create multiple objects with the same structure.
- **Instance Properties :**
 - **Instance properties** are the variables (data) that belong to a **specific object created from a constructor**.
 - Every time you create an object using `new`, that object gets its own copy of those properties.
- `Error.prototype.toString()` :
The **toString()** method of [Error](#) instances returns a string representing this error.
- **Error cause :**
Helps in understanding what went wrong and why it went wrong, even if one error is created because of another error.

Syntax: `throw new Error("Main error message", { cause: "Extra details here" });`

We use cause because error “chain” together:

A fetch fails.

A database save fails

A fire read fails

instead of losing the original error, cause lets you **keep it**.

Cause help's in:

1. Debugging complex systems
2. preserves original error
3. makes logs easier to understand
4. Standard features

5. Concept

Meaning

err.message

The main error

err.cause

The underlying reason

Error Column Number :

When JavaScript throws an error, it often shows:

- **Line number** → which line the error happened
- **Column number** → which *exact position* in that line caused the error

Example error :

Uncaught SyntaxError: Unexpected token)
at script.js:12:18

Here:

- 12 → **line number**
- 18 → **column number**

Column number means the error happened at the **18th character** of line 12.

Term

Meaning

Line number

Which line error happened

Column
number

Which exact character in that line

Helps with

Debugging, finding syntax
problems.

- Error message :

Property	Meaning
err.message	The main text describing the problem
err.name	Type of error (Error, TypeError, etc.)
err.stack	Full trace (filename, line, column

- FUNCTIONS :

A function is a block of code that performs a task and can be reused again and again

A list of parameters to the function, enclosed in parentheses and separated by commas.

Function Expressions :

- ✓ Can be stored in variables
- ✓ Can be passed as arguments (callbacks)
- ✓ Can be returned from functions
- ✓ They are **not hoisted** like function declarations

Type	Hoisted?	Can you call it before definition?
Function Declaration	Yes (fully)	Yes
Function Expression	Variable only	No
Arrow Function	Variable only	No

- Hoisting Of Variables :

Var is hoisted and initialized with undefined

Let and const are hoisted but not initialized, this creates Temporal dead zone.

Example :

hello();

```
function hello() {  
  console.log("Hi");  
}
```

Diagram:

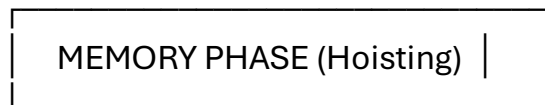
Memory (Hoisted)

hello: function {...}

Execution

hello();

Diagram for the function hoisting :



Function Declarations:

greet → function() {...}

var Variables:

var x → undefined

let / const Variables:

let y → TDZ (uninitialized)

const z → TDZ (uninitialized)

EXECUTION PHASE

Code runs line-by-line:

`greet();` ✓ works — function already hoisted

`x;` ✓ exists but is undefined

`y;` ✗ ReferenceError (TDZ)

`z;` ✗ ReferenceError (TDZ)

- Recursion :

Recursion is when a function calls itself to solve a smaller version of the same problem.

Think of it like:

- You break a big task into small tasks
- Each small task is solved the same way
- Eventually, you reach a point where you don't need to break it anymore → **base case**
- **Base Case (stop condition)**
The condition where the function stops calling itself.
Without it → infinite loop → stack overflow.
- **Recursive Case**
The part where the function calls itself with **smaller input**.

DIAGRAM EXPLAINING RECURSION

Recursion

|

| calls itself with smaller input

v

function(x):

```

if x is small enough:
    return result    <-- Base Case
else:
    return function(x-1) <-- Recursive Case

```

- **IIFE = Immediately Invoked Function Expression**

It means:

1. You create a function
2. AND you call it **immediately**
3. Without using the function name separately

Basic form of an IIFE

```

(function() {
    console.log("I am an IIFE!");
})();

```

Breakdown:

- (function() { ... }) → Function Expression
- () → Immediately calls that function

Example :

```

( function() {
    // code
}) ();

```

↑ ↑

function calling immediately
expression

- **Concept**

IIFE vs
Normal
Function

What it means

IIFE runs immediately
and doesn't pollute
global scope

Real-
world use

Initialization, privacy, older
module systems

Return
values

IIFE can return data,
objects, or functions

Module
Patterns

Used to build
private/public modules
before ES6

- Closure :
- A **closure** is created when:

A function **remembers** the variables.
From the **place where it was created**.
Even after that outer function has finished running.

A function + the environment it remembers = closure

Real Diagram :

outer()

└─ x = 10 (local variable)
└─ returns inner()

inner() <-- still remembers x

- Function Parameters :
Function parameters are placeholders (variables) listed inside the parentheses of a function definition.

Example:

```
function greet(name) {  
  console.log("Hello " + name);  
}
```

Function call :

```
greet("Manu");
```

Diagram:

```
function greet( name ) {  
    ↑  
    parameter (empty variable)  
}
```

```
greet( "Manu" );  
    ↑  
    argument (value you pass)
```

- Arrow Functions:
Arrow functions are a **shorter, cleaner way** of writing functions in JavaScript.

Example:

Normal function:

```
function add(a, b) {  
    return a + b;  
}
```

Arrow function:

```
const add = (a, b) => a + b;
```

- ERRORS :
- Aggregate Error:

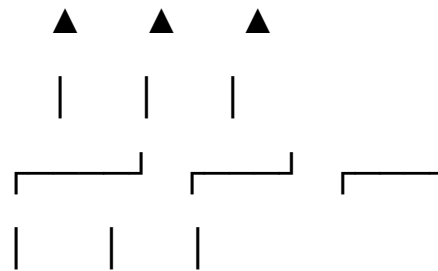
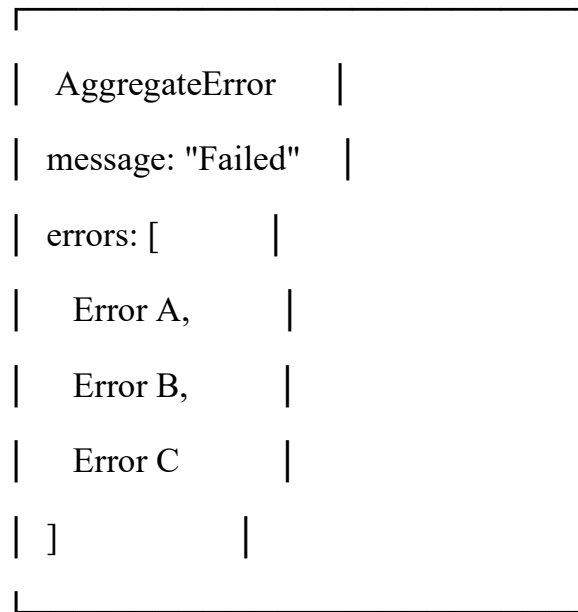
`AggregateError` is a special type of error in JavaScript that groups **multiple errors into one single error object**.

It is mainly used with **Promise.any()**, but you can also create it manually.

Uses:

When multiple things run at the same time, more than one can fail. `AggregateError` lets you capture ALL failures together instead of just the first one.

Diagram (Easy to Visualize):



Error A Error B Error C